Linear Data Structures: Arrays and Memory Representation

Arrays provide contiguous memory allocation where each element can be accessed in O(1) time using index calculation: Address(A[i]) = BaseAddress + i * sizeof(Element). Because elements are stored contiguously in cache lines, arrays exhibit exceptional spatial locality of reference, minimizing CPU cache misses during sequential iteration.

Dynamic arrays (such as Python lists or C++ std::vector) allocate extra capacity upon initialization. When current capacity is exceeded, the dynamic array allocates a new memory block with double the previous capacity, copies all existing elements over, and deallocates the old memory. While this reallocation requires O(n) time, amortized analysis proves that across n insertions, the aggregate cost is O(n), yielding an O(1) amortized insertion runtime.

Linked Lists: Singly vs Doubly Linked Structures

Linked lists store discrete node structures connected via pointer references. Each node in a singly linked list contains data and a next pointer, requiring pointer memory overhead. Doubly linked lists store both next and prev pointers, allowing bidirectional traversal at the cost of additional memory per node.

Advantages: Insertions and deletions at the head or known positions execute in O(1) time without contiguous memory reallocation. Disadvantages: Direct indexing requires O(n) sequential traversal, and scattered node memory locations cause frequent CPU cache line invalidations compared to contiguous arrays.

Stacks, Queues, and Asymptotic Complexity Analysis

Stacks enforce Last-In, First-Out (LIFO) semantics with push and pop operations executing in O(1) time. Stacks govern procedure call execution frames, recursive backtracking, and syntax tree parsing.

Queues enforce First-In, First-Out (FIFO) semantics, utilized in task scheduling, asynchronous message buffers, and Breadth-First Search (BFS). Circular ring buffers avoid O(n) element shifting during dequeue.

Asymptotic Notation: Big-O represents the upper bound asymptotic complexity, Big-Omega represents the lower bound, and Big-Theta represents a tight bound where f(n) is bounded above and below by constants c1*g(n) and c2*g(n).